

Utilizarea sistemului de conversie analog-digitala si a depanatorului

1 Scopul lucrarii

Studentul ar trebui ca, la sfarsitul sedintei de laborator, sa aiba cunostinte de baza despre functionarea sistemului de conversie analog-digitala intern al chip-ului C8051F040 si sa utilizeze practic (in programe asm) registrele speciale ale acestui sistem. De asemenea, studentul isi va insusi modul de utilizare al programului Silicon Laboratories IDE (Integrated Development Environment) pentru operatia de depanare.

2 Introducere

In acest capitol vor fi prezentate:

- 2.1. Sistemul de conversie analog-digitala.
- 2.2. Silicon Laboratories IDE - Depanatorul.

2.1 Sistemul de conversie analog-digitala (ADC2)

Microcontrolerul C8051F040 dispune de doua sisteme de conversie analog-digitala, unul pe 12 biti, numit ADC0 (analog-digital convertor 0) si un altul pe 8 biti, numit ADC2. Platforma isi propune sa detalieze functionarea sistemului de conversie pe 8 biti.

Subsistemul ADC2 este format dintr-un multiplexor analogic cu 8 canale, un amplificator programabil si un convertor analog-digital. Convertorul contine un registru de aproximari successive (SAR) pe 8 biti si un circuit intern de esantionare si memorare (sample&hold), iar viteza de achizitie este de 500ksps (kilo samples per second). ADC2 foloseste de asemenea si referinta de tensiune. Schema bloc a sistemului, impreuna cu registrele speciale specifice lui sunt prezentate in Fig. 2.1.

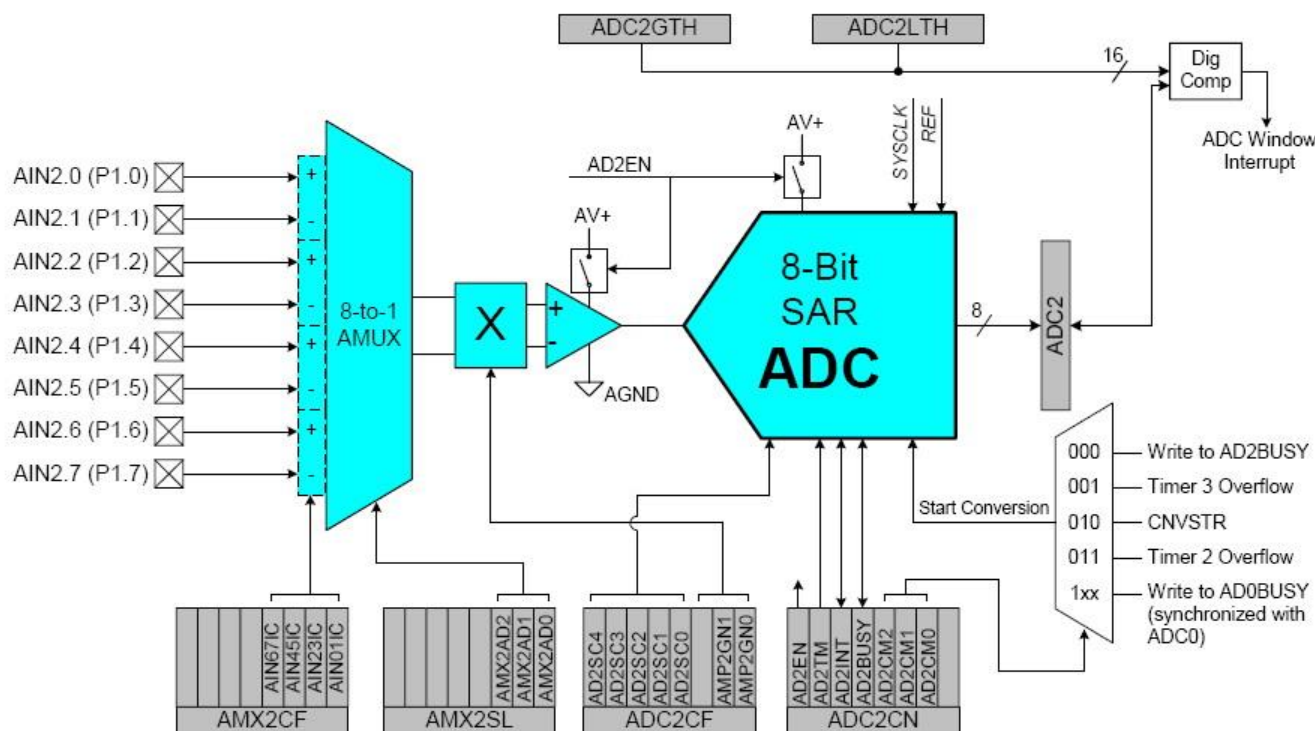


Fig. 2.1. Schema bloc a sistemului de conversie analog-digitala

Canalele multiplexorului, amplificarea, si modul de conversie a datelor sunt programabile din software prin intermediul registrelor speciale prezentate de asemenea in Fig. 2.1.

Intreg sistemul functioneaza numai atunci cand bitul de enable este setat, adica atunci cand bitul AD2EN (ADC2CN.7) are valoarea logica 1.

2.1.1 Multiplexorul analogic si amplificatorul programabil

Dupa cum se observa pe schema bloc a sistemului, rolul multiplexorului analogic este acela de a selecta, la intrarea amplificatorului programabil, unul dintre cele opt canale analogice sau diferenta a 2 canale vecine. Utilizatorul are posibilitatea de a modifica prin software pinii de la care se citește tensiunea analogica cat si modul de achizitie al acesteia: simplu (single-ended) sau diferential. Doua registre speciale definesc functionarea multiplexorului analogic: AMX2CF (AMUX2 configuration register) si AMX2SL (AMUX2 channel select). Bitii 0, 1, 2 si 3 ai registrului AMX2CF corespund celor patru perechi de pini de intrare si configureaza fiecare pereche ca fiind de tip diferential (daca sunt setati) sau de tip single-ended (daca nu sunt setati). Diferitele combinatii ale bitilor 0, 1 si 2 ai registrului AMX2SL selecteaza o pereche diferentiala sau un singur pin la iesirea multiplexorului, conform Tabelului AMUXSelChart. Pentru mai multe informatii despre definitia AMX2CF, AMX2SL si Tabelul AMUXSelChart, consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 95, pag. 96).

Important: Pinii de intrare in multiplexor sunt si pinii de intrare/iesire (I/O) ai portului P1. Pentru a fi disponibili intr-o forma adecvata la intrarea multiplexorului ei trebuie setati ca pini de intrare analogica, adica bitii corespunzatori din registrul special P1MDIN sa fie setati la valoarea logica 0.

Amplificatorul programabil are rolul de a amplifica tensiunea de la intrare cu un anumit castig fix, programat prin software. Castigul poate fi 0.5, 1, 2 sau 4. In mod implicit el este 0.5, iar celelalte valori pot fi programate modificand bitii 0 si 1 ai registrului special ADC2CF. Pentru mai multe informatii despre definitia ADC2CF consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 97).

2.1.2 Modurile de operare ale convertorului

ADC2 are o viteza maxima de conversie de 500ksps. Tactul de conversie al ADC2 este derivat din tactul sistemului. Raportul lor este setat prin software prin bitii AD2SC (cei mai semnificativi 5 biti) ai registrului special ADC2CF. Totusi, tactul ADC2 nu poate depasi 7.5MHz.

O conversie poate fi initiata in unul dintre urmatoarele cinci moduri, in functie de valorile bitilor AD2CM2-0 (bitii 1, 2 si 3) ai registrului special ADC2CN:

- prin setarea bitului AD2BUSY din registrul special ADC2CN;
- prin depasirea timerului Timer3;
- prin aparitia unui front crescator la pinul extern de initiere de conversie (CNVSTR2 sau CNVSTR0);
- prin depasirea timerului Timer2;
- prin setarea bitului AD0BUSY din registrul special ADC0CN (aceasta duce la initierea simultana, printr-o singura comanda, a conversiei atat pentru ADC0, cat si pentru ADC2).

In aceasta lucrare vom folosi cel mai simplu mod de a initia o conversie si anume setarea bitului ADC2BUSY. Pentru mai multe informatii despre celelalte moduri de operare consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 92).

In timpul conversiei bitul AD2BUSY ramane setat (la 1), urmand sa se reseteze (la 0) atunci cand conversia se va fi terminat. Frontul cazator al acestui bit genereaza o intrerupere (daca acestea sunt activate) si seteaza flag-ul de intrerupere din ADC2CN (bitul 5). Datele convertite sunt disponibile in registrul special ADC2. Procedura de determinare a sfarsitului unei conversii, atunci cand aceasta este initiata prin setarea bitului ADC2BUSY este prezentata in cativa pasi simpli:

- Pasul 1. Resetati bitul AD2INT(valoarea logica 0);
- Pasul 2. Setati bitul AD2BUSY(valoarea logica 1);
- Pasul 3. Asteptati pana cand bitul AD2INT isi schimba valoarea logica in 1;
- Pasul 4. Procesati datele aflate in ADC2.

Pentru mai multe informatii despre definitia ADC2CF, definitia ADC2CN si definitia ADC2 consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 97, pag. 98, pag. 99).

2.1.3 Referinta de tensiune

Circuitul intern de referinta de tensiune ofera flexibilitate maxima in operarea cu sistemele de conversie analog-digitala si digital-analogica. Exista trei pini externe care ofera posibilitatea ca fiecare ADC si cele doua DAC-uri sa aiba referinte diferite (externe sau interne) de tensiune. ADC0 poate de asemenea sa aiba ca referinta de tensiune iesirea DAC0, iar ADC2 poate avea ca referinta de tensiune tensiunea de alimentare, dupa cum se observa in Fig. 2.1.3.

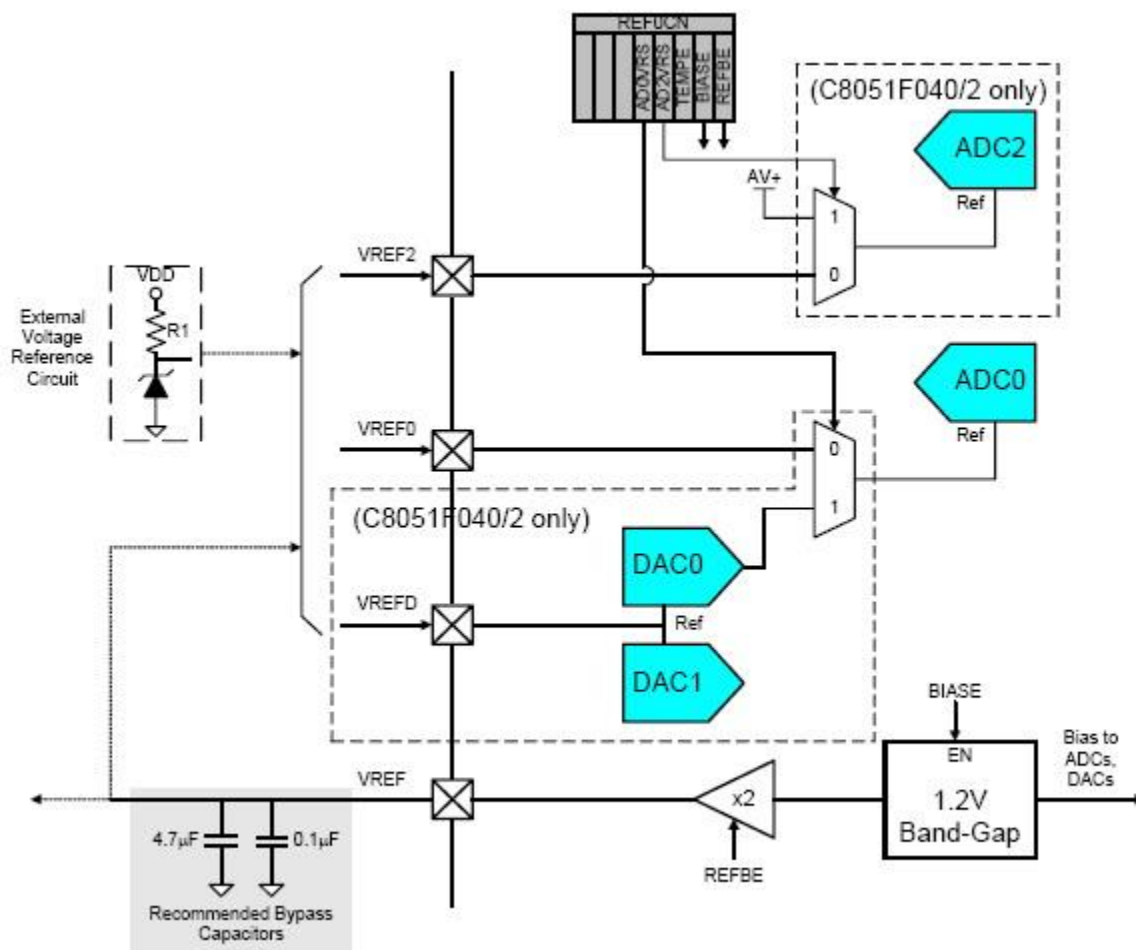


Fig. 2.1.3. Circuitul de referinta de tensiune

Circuitul de referinta interna de tensiune este format dintr-un generator de 1,2V cu o stabilitate buna cu temperatura si un amplificator buffer cu castigul de 2. Referinta interna poate fi rutata prin intermediul pinului VREF catre alte sisteme exterioare, sau catre pinii de intrare ai tensiunii de referinta din Fig. 2.1.3. (VREF0, VREF2, VREFD). Se recomanda folosirea unor condensatoare de suntare de la pinul VREF catre masa, asa cum se observa in Fig. 2.1.3.

Circuitul de referinta de tensiune este configurat printr-un singur registru special: REF0CN. Acest registru este raspunzator de activarea si dezactivarea generatorului intern de referinta si de selectarea referintei de tensiune pentru cele doua convertoare analog-digitale. Bitul 0, BIAS, activeaza generatorul de tensiune stabilizata, iar bitul 1, REFBE, activeaza amplificatorul cu castig 2. BIAS trebuie sa fie setat chiar daca nu se foloseste referinta interna de tensiune, dar se foloseste unul dintre convertoare. Daca se doreste folosirea referintei interne de tensiune pentru convertoare sau pentru vreun alt sistem extern, atunci ambii biti (BIAS si REFBE) trebuie sa fie setati la 1 logic. Bitii 4 si 5, AD0VRS si AD2VRS selecteaza tensiunile de referinta pentru ADC0, respective ADC2.

Pentru mai multe informatii despre definitia REF0CN consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 114).

2.2 Silicon Laboratories IDE – Depanatorul

Majoritatea microcontrolerelor sunt prevazute cu o interfata JTAG si cu logica necesara pentru a suporta o serie de operatii folosite in procesul de productie si testare. Astfel, prin intermediul acestei interfete, un modul exterior microcontrolerului (un sistem programabil care poate fi si un PC) are posibilitatea de a scrie si citi memoria de program (Flash), memoria interna sau externa de date (RAM) si, implicit, registrele de uz general sau speciale.

Aplicatia Silicon Laboratories IDE foloseste acest modul in operatiile proprii de “Download Code”, “Memory Fill”, “Erase code space”, “Upload Memory to File” si pentru a tine in permanenta evidenta valorile din memoria de date (valori pe care utilizatorul le foloseste efectiv la depanare). In continuare, vor fi prezentate cateva din tool-urile auxiliare mentionate mai sus si principalele functii ale depanatorului.

2.2.1 Conectarea placii de dezvoltare la PC

Nici unul din tool-urile prezentate in continuare si nici functiile depanatorului nu pot fi exemplificate decat conectand in prealabil placa de dezvoltare (si implicit microcontrolerul) la calculator, deoarece Silicon Laboratories IDE nu este un simulator. Executia programelor se face direct pe hardware-ul atasat si nu virtual.

Se realizeaza intai conexiunea hardware urmand instructiunile prezentate in Lucrarea nr. 1, capitolul 2.2. Porniti, apoi, aplicatia Silicon Laboratories IDE (versiunea 2.82).

Din meniul Options -> Connection Options configurati conexiunea astfel:

1. Selectati optiunea Serial Adapter - USB Debug Adapter.
2. Bifati Serial Adapter - Power target after disconnect.
3. Selectati Debug Interface - JTAG.

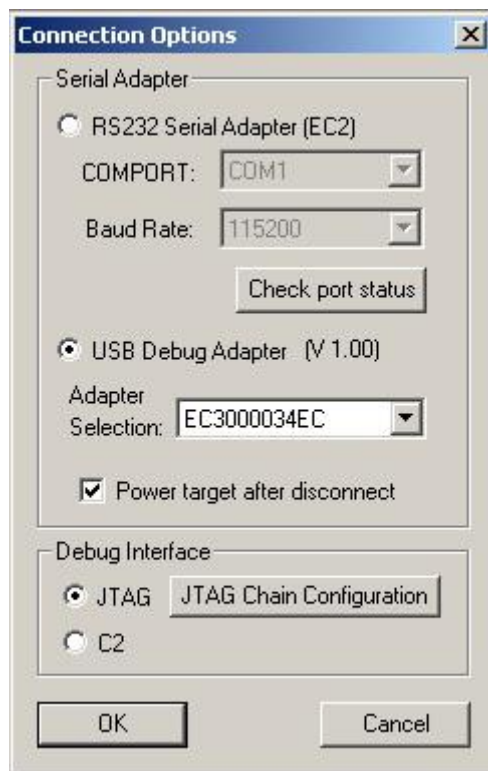


Fig. 2.2.1. Connection Options

Initiati conexiunea cu placa de dezvoltare apasand butonul Connect din bara de butoane, sau selectand meniul Debug -> Connect.

2.2.2 Tool-urile Code/RAM memory fill si Erase code space

Pentru a vizualiza memoria RAM deschideti fereastra Ram Viewer apasand butonul Ram Window din bara de butoane sau selectand meniul View -> Debug Windows -> RAM. Analizand aceasta

ferestra, observati ca in coloana din stanga sunt plasate adrese ale memoriei RAM. Coloanele din centrul ferestrei reprezinta valorile stocate in memoria RAM la adresele corespunzatoare din stanga, iar coloanele din dreapta reprezinta caracterele ASCII corespunzatoare valorilor stocate. Memoria RAM poate fi modificata simplu utilizand aplicatia RAM Memory Fill selectabila din meniul Tools -> Memory Fill -> RAM. Completati campurile Address Range Selection - Starting Address si Address Range Selection - Ending Address, tinand cont ca adresa de final trebuie sa fie mai mare (ca numar) decat adresa de inceput. Introduceti o valoarea cu care doriti sa umpleti memoria RAM intre cele doua adrese completate anterior in campul Patern Selection - Byte for Fill. Apasati butonul Fill si urmariti modificarile in fereastra Ram Viewer. Tocmai ati modificat memoria RAM interna microcontrolerului!

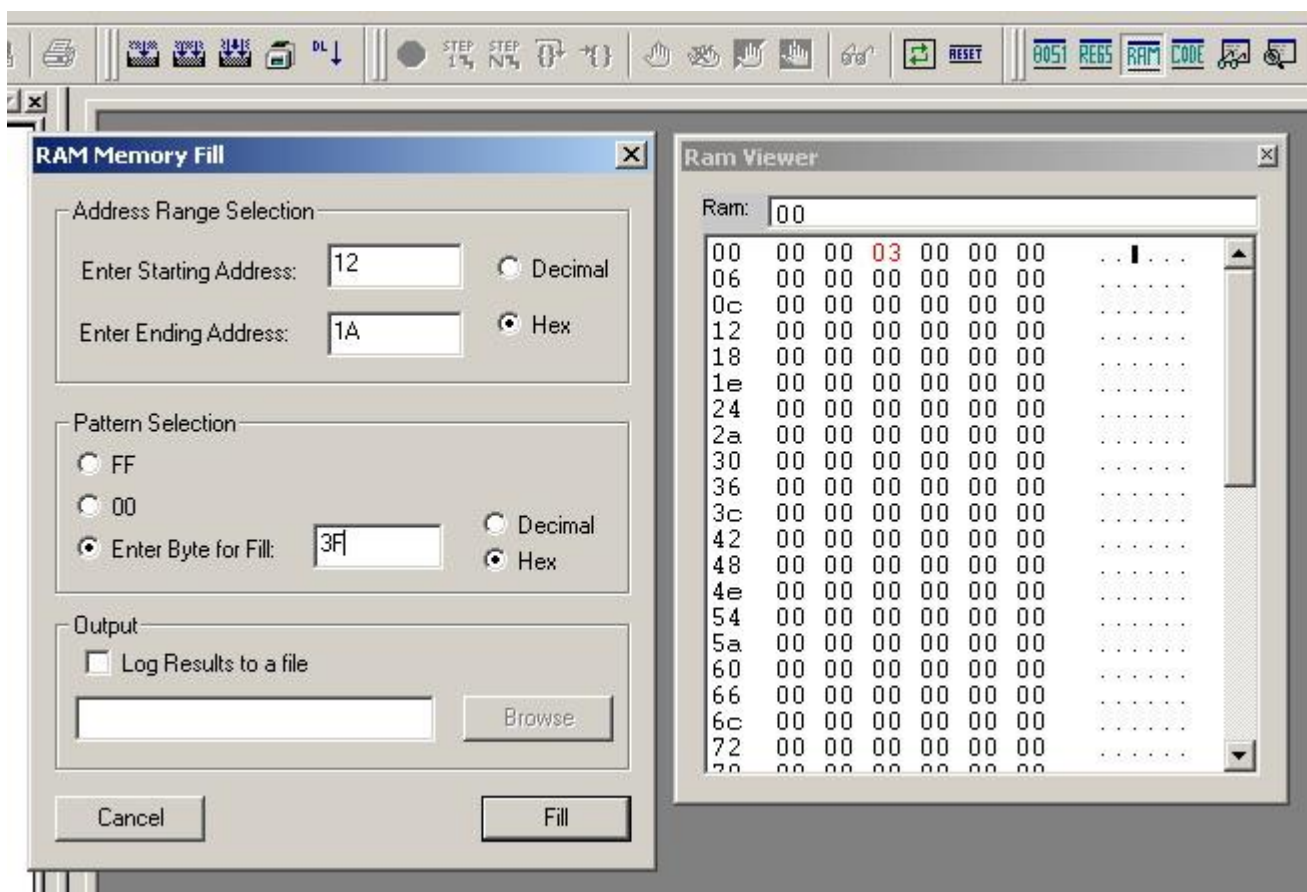


Fig. 2.2.2.a. Tool-ul RAM Memory Fill si fereastra Ram Viewer

In acelasi mod in care a fost modificata memoria RAM interna a microcontrolerului, poate fi manipulata si memoria de program (Flash). Pentru a vizualiza memoria de program deschideti fereastra Memory Viewer apasand butonul Code Window din bara de butoane sau selectand meniul View -> Debug Windows -> Code Memory. Analizand aceasta fereastra, observati ca in coloana din stanga sunt plasate adrese ale memoriei Flash. Coloanele din centrul ferestrei reprezinta valorile stocate in memoria Flash la adresele corespunzatoare din stanga, iar coloanele din dreapta reprezinta caracterele ASCII corespunzatoare valorilor stocate. Memoria Flash poate fi modificata simplu utilizand aplicatia Code Memory Fill selectabila din meniul Tools -> Memory Fill -> Code. Completati campurile Address Range Selection - Starting Address si Address Range Selection - Ending Address, tinand cont ca adresa de final trebuie sa fie mai mare (ca numar) decat adresa de inceput. Introduceti valoarea cu care doriti sa umpleti memoria Flash intre cele doua adrese completate anterior in campul Patern Selection - Byte for Fill. Apasati butonul Fill si urmariti modificarile in fereastra Memory Viewer. Tocmai ati modificat memoria Flash a microcontrolerului!

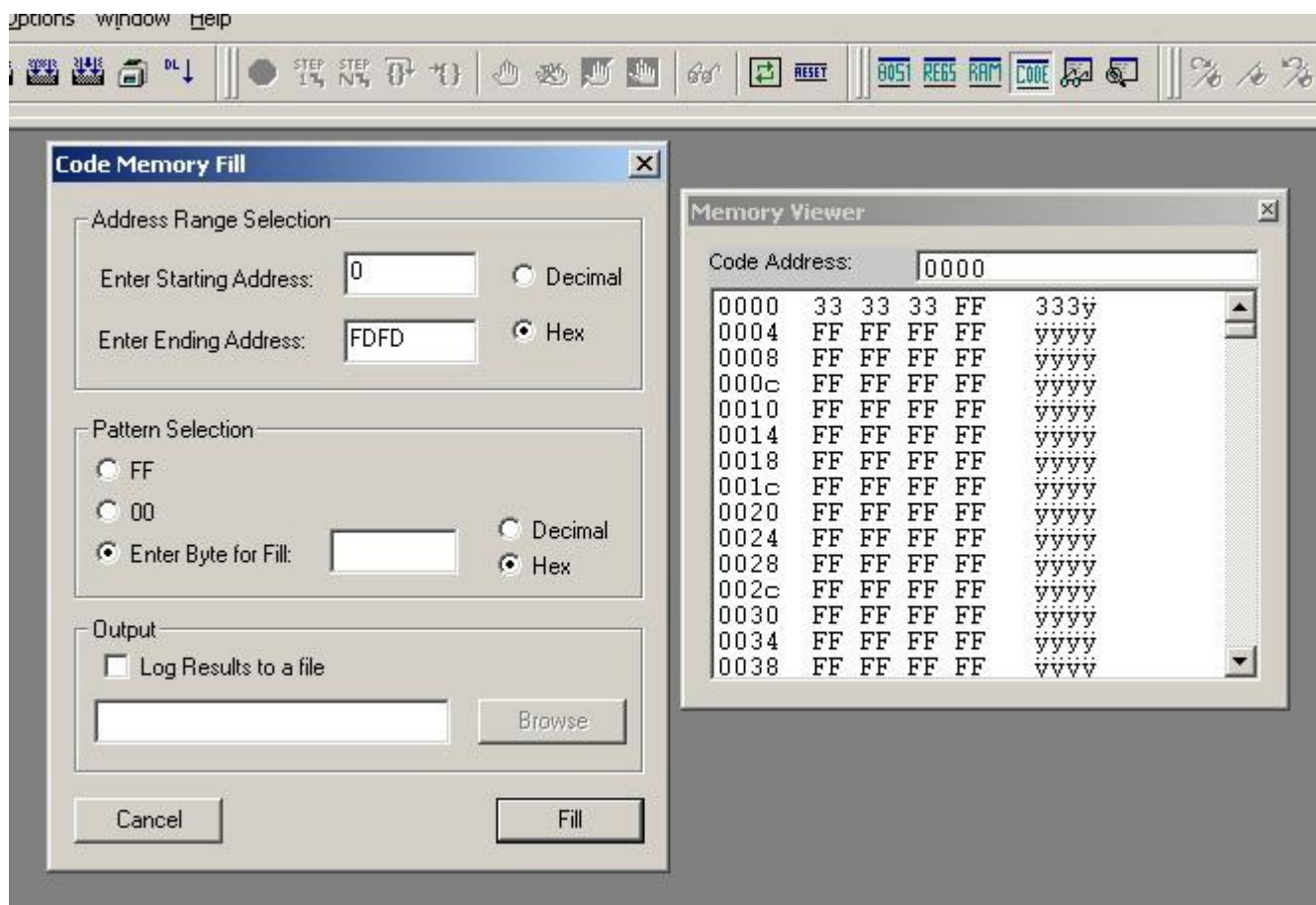


Fig. 2.2.2.b. Tool-ul Code Memory Fill si fereastra Memory Viewer

Memoria Flash este un tip special de memorie cu urmatoarele particularitati:

- este organizata in pagini de cate 512 octeti;
- nu poate fi suprascrisa la nivel de byte decat prin stergere, adica prin trecerea bitilor cu valoarea 1, in biti cu valoare 0;
- trecerea bitilor cu valoare 0 in biti cu valoare 1 se poate face doar prin setarea tuturor bitilor dintr-o pagina.

Aplicatia Silicon Laboratories IDE contine, pe langa tool-ul Code Memory Fill care manipuleaza memoria Flash la nivel de byte si un alt tool care reseteaza memoria Flash (umple memoria cu valoarea FF) – manipulare la nivel de pagina. Dupa acest reset, memoria Flash poate fi scrisa normal pentru ca orice operatie de scriere va insemna trecerea unor biti din 1 in 0, operatie care este posibila dupa cum este prezentat mai sus. Deschideti fereastra Memory Viewer pentru a vizualiza valorile curente din memoria Flash. Resetati memoria Flash selectand meniul Tools -> Erase Code Space. Observati ca toti octetii memoriei de program au fost setati la valoarea FF.

2.2.3 Depanatorul

Nota: Functiile depanatorului nu sunt active decat in momentul in care placa de dezvoltare este conectata la calculator si in microcontroler a fost scris un program. Exemplificarile functiilor prezentate in continuare vor fi realizate in capitolul 4.

Executia programului curent se porneste/opreste apasand butonul Go/Stop aflat in bara de butoane a IDE-ului.

Pentru o executie pas cu pas se foloseste butonul Step imediat adiacent.

Executia pas cu pas poate fi facuta si cu o granularitate mai mare a pasilor utilizand butonul Multiple Step. Configurarea acestui tip de executie se realizeaza selectand meniul Options -> Multiple Step Configuration.... In fereastra de dialog se poate seta numarul de instructiuni care se

vor executa la un pas, cat si tipul de refresh intre pasi (doar numaratorul de program PC sau si ferestrele de debug).

Butonul imediat adiacent, *Step over*, ofera posibilitatea trecerii peste instructiunea urmatoare fara a o executa efectiv.

Exista doua procedee prin care programul poate fi executat pana la o anumita instructiune. Butonul *Run to cursor* realizeaza acest lucru fara a impune nici o restrictie privind o executie ulterioara a programului. Cursorul se pozitioneaza pe linia instructiunii inaintea executiei careia se doreste oprirea, apoi se apasa butonul precizat. Cel de-al doilea procedeu, mai automatizat, este utilizarea breakpoint-urilor. IDE-ul ofera posibilitatea setarii de breakpoint-uri pe orice linie din program ce reprezinta o instructiune apasand butonul *Insert/Remove Breakpoints*. Efectul este acelasi: executia programului se opreste in momentul intalnirii respectivului breakpoint. Ulterior breakpoint-ul poate fi eliminat apasand acelasi buton *Insert/Remove Breakpoints* sau doar dezactivat apasand butonul *Enable/Disable Breakpoint*. Reactivarea unui breakpoint se face apasand din nou butonul mai sus mentionat. Managementul breakpoint-urilor se poate face si cu click dreapta pe instructiunea asprura careia se doreste sa se actioneze.

Toate functionalitatile prezentate mai sus pot fi gasite si in meniul *Debug*. Aceste functionalitati sunt deosebit de utile in cazul aparitiei de erori in program si a necesitatii depanarii codului sursa prin vizualizarea si modificarea memoriilor de program si de date si a registrelor speciale in timp real. Acestea nu pot fi vizualizate si modificate decat intr-o pauza a executiei, deci nu sincron cu aceasta.

Cele mai importante informatii despre un program in executie sunt oferite prin intermediul ferestrelor de debug. Aceste ferestre grupeaza registrele cu functii speciale pe categorii permitand vizualizarea si modificarea lor. Datele nu sunt actualizate in timpul executiei programului, ci numai intr-o pauza a executiei. Vom prezenta in continuare cateva dintre aceste ferestre de debug:

1. Fereastra 8051 Controller

- se deschide apasand butonul *8051 SFR's* din bara de butoane sau selectand meniul *View -> Debug Windows -> SFR's -> 8051 Controller/Misc*.
- cele mai importante registre speciale continute:
 - PC (program counter) – registru ce retine adresa instructiunii curente.
 - SP (stack pointer) – registru ce indica adresa ultimului element din stiva.
 - DPTR (data pointer) – registru pe 16 biti folosit ca pointer la memoria RAM externa (64kB) sau la memoria de program (64kB).
 - PSW (program status word) – registru ce retine starea curenta a programului. Contine informatii despre bancul de registre curent, flag-urile de carry si overflow, etc. Pentru mai multe informatii despre definitia PSW consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 152).
 - ACC (accumulator) – registru acumulator principal.
 - B – registru acumulator secundar. Este folosit in operatii complexe impreuna cu acumulatorul ACC.
 - SFRPAGE – registru ce indica pagina curenta de registre speciale. Pentru mai multe informatii despre definitia SFRPAGE consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 142).

2. Fereastra registrelor de uz general

- se deschide apasand butonul *Register Window* din bara de butoane sau selectand meniul *View -> Debug Windows -> Registers*.
- contine cele opt registre generale din bancul curent de registre: R0, R1, ..., R7.

3. Fereastra stivei

- se deschide selectand meniul *View -> Debug Windows -> Stack*.
- contine:
 - SP (stack pointer) – registru ce indica adresa ultimului element din stiva.
 - LIMIT – limita superioara a stivei. Daca se incearca introducerea in stiva peste aceasta limita apare overflow.
 - COUNT – indica numarul octeti aflati la acel moment in stiva.

- STACK CONTENTS – indica continutul din acel moment al stivei. Prima coloana indica adresa la care se afla acel element, a doua elementul respectiv (in hexa), iar a treia o descriere a elementului respectiv.
4. Fereastra porturilor
- se deschide selectand meniul View -> Debug Windows -> SFR's -> Stack.
 - cele mai importante registre speciale continute:
 - Px (x = 0..7) – cele opt porturi ale microcontrolerului.
 - PxMDOUT (x = 0..7) – opt registre de configurare fiecare corespunzand cate unui port. Aceste registre configureaza portul fizic (open-drain sau push-pull).
 - PxMDIN (x = 1..3) – trei registre de configurare corespunzand porturilor 1, 2 si 3. Aceste registre seteaza portul ca intrare analogica sau digitala.
5. Fereastra sistemului de conversie analog-digitala ADC2
- se deschide selectand meniul View -> Debug Windows -> SFR's -> ADC2.
 - cele mai importante registre continute sunt:
 - AMX2CF – registrul de configurare al multiplexorului analogic.
 - AMX2SL – registrul de selectie al multiplexorului analogic.
 - ADC2CN – registrul de control al sistemului de conversie analog-digitala.
 - ADC2CF – registrul de configurare al sistemului de conversie analog-digitala.
 - ADC2 – registru ce retine valoarea rezultata in urma unei conversii.

Pentru mai multe informatii in legatura cu aceste registre consultati capitolul 2.
Toate ferestrele de depanare mentionate mai sus sunt prezentate in Fig. 2.2.3.

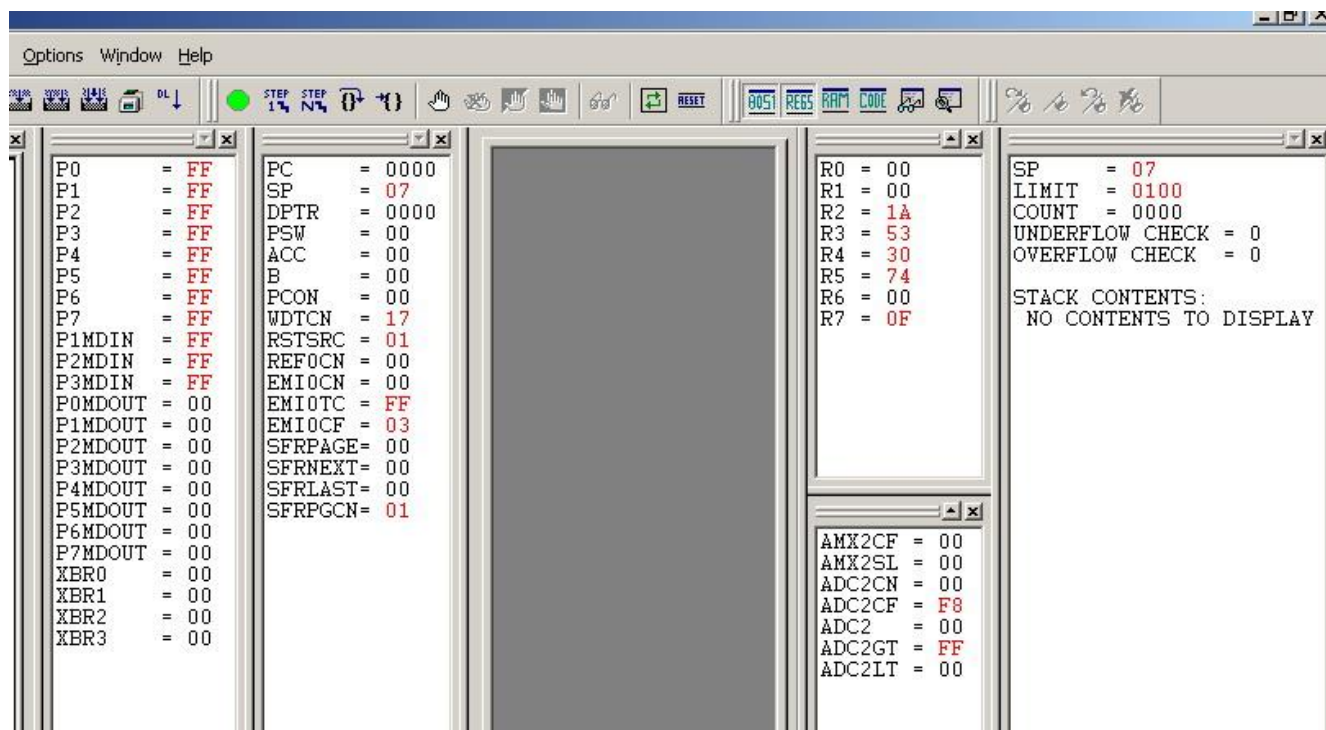


Fig. 2.2.3. Principalele ferestre de depanare

3 Problema

3.1 Formularea problemei

Se cere sa se implementeze un sistem automat de masurare si indicare a intensitatii luminoase dintr-o incinta. Pentru indicarea intensitatii luminoase se va folosi o metoda vizuala calitativa.

3.2 Solutie posibila

Nota: o alta solutie posibila, ce implica un algoritm mai pretentios este prezentata in ANEXA 5.4.

3.2.1 Descrierea modului hardware

Vom prezenta in continuare o implementare a unui sistem avand la baza placa de dezvoltare cu microcontrolerul C8051F040. Acesteia i se adauga doua module externe:

- modulul de intrare
 - contine o celula solara (dispozitivul de achizitie) ce se comporta ca o sursa de tensiune variabila (tensiunea oferita la iesirea celulei solare este direct proportionala cu intensitatea luminii incidente).
 - se conecteaza la microcontroler prin doua fire; unul este masa sistemului, iar celalalt este pentru tensiunea de iesire a celulei solare.
 - schema modului este prezentata in Fig. 3.2.1.

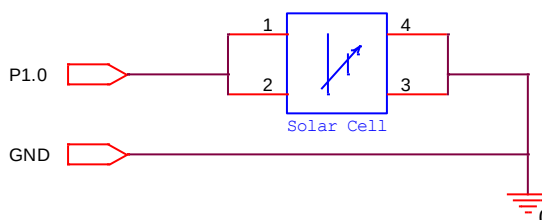


Fig. 3.2.1. Modulul de intrare

- modulul de iesire
 - contine, printre alte componente ce nu fac obiectul acestei lucrari, o bara cu zece led-uri.
 - se conecteaza la microcontroler prin doua magistrale cu zece fire; se folosesc opt fire fire dintr-o magistrala si doua fire din cea de-a doua magistrala.
 - modul de conectare al fiecarui led la microcontroller este prezentat in ANEXA 5.2.

Dupa cum am spus celula solara se comporta ca o sursa de tensiune variabila cu valori intre 0 si 2.4 volti. Aceasta tensiune este introdusa in multiplexorul analogic prin pinul P1.0, modul de achizitie fiind simplu (nu diferential).

Valoarea digitala de la iesirea sistemului de conversie analog-digitala este data de formula:

$$\text{Cod} = V_i * (K/V_{\text{ref}}) * 256$$

In care:

- Cod este pe opt biti (intre 0 - 255 in zecimal)
- V_i este tensiunea de la intrarea multiplexorului analogic
- K este castigul amplificatorului programabil (0.5, 1, 2, 4)
- V_{ref} este tensiunea de referinta

Observatie: codul convertit este un numar intre 0 si 255, deci $V_i * (K/V_{\text{ref}})$ trebuie sa aiba o valoare subunitara.

In aceste conditii se alege tensiunea de referinta V_{ref} egala cu 2.4V (se foloseste chiar referinta interna de tensiune a microcontrolerului), iar castigul amplificatorului programabil K egal cu 1. Pentru a obtine 2.4V la iesirea referintei interne de tensiune trebuie activat si amplificatorul de castig 2 (vezi capitolul 2.1.3.). Pentru a ruta referinta de tensiune pinii 5 si 6 din J22 trebuie conectati printr-un jumper.

La iesirea sistemului se conecteaza pe pinii P2.5, P2.7, P3.0 – P3.7 zece led-uri grupate pe o bara de led-uri. Sistemul va aprinde mai multe sau mai putine led-uri de pe aceasta bara in functie de intensitatea luminoasa captata de celula solara: intensitate luminoasa mica – putine led-uri aprinse, intensitate luminoasa

mare – multe led-uri aprinse. Putem semnala astfel 11 grade de intensitate luminoasa: de la niciun led aprins pana la toate cele 10 led-uri aprinse.

3.2.2 Descrierea algoritmului

Observatie: programul respecta structura generala prezentata in Lucrarea nr.1, capitolul 3.2.2.

In solutia propusa in acest laborator programul va avea in mai multe rutine: `main`, `init`, `lightTest`, `lightLedBar`.

Algoritmul consta intr-o bucla ce se autoapeleaza la infinit. Aceasta bucla are doua etape:

- se convertește tensiunea de la bornele celulei solare într-o valoare digitală pe opt biți (între 0 și 255); aceasta valoare este împartită la 24 pentru a obține un număr între 0 și 10, reprezentând numărul de led-uri ce vor trebui să fie aprinse,
- se aprind un număr de led-uri de pe bara de led-uri în funcție de numărul obținut la pasul anterior.

Subrutina `lightTest` face o conversie analog-digitală. Valoarea rezultată (între 0 și 255) este împartită la 24, obținându-se un număr între 0 și 10, reprezentând numărul de led-uri ce vor trebui să fie aprinse. Acest număr este stocat în registrul acumulator A. A se nota faptul că instrucțiunea `div AB` împarte valoarea din A la valoarea din B și stochează rezultatul astfel: catul în A, restul în B.

Rutina principală (`main`) apelează subrutina de inițializare (`init`), apoi trece în zona de buclă infinită etichetată `start`. Aici se apelează rutina `lightTest` și apoi, utilizând valoarea din A ca și parametru, se apelează rutina `lightLedBar`. În final, se execută un salt necondiționat la eticheta `start`.

Subrutina de inițializare

- dezactivează întreruperile
 - pentru detalii, a se vedea explicațiile din Lucrarea nr. 1.
- dezactivează watchdog timer-ul
 - pentru detalii, a se vedea explicațiile din Lucrarea nr. 1.
- activează crossbar-ul
 - pentru detalii, a se vedea explicațiile din Lucrarea nr. 1.
- configurează portul P1
 - aplicația configurează pinul 0 din portul P1 ca fiind pin de intrare analogică pentru convertorul analog-digital. Configurarea modului de intrare al portului se face modificând valoarea stocată în registrul P1MDIN.
 - definiția registrului P1MDIN se află în documentația microcontrolerului (fișierul C8051F04xRev1_4.pdf, pag. 219).
- configurează portul P2 și portul P3
 - aplicația folosește biții 5 și 7 ai portului P2 și toți biții portului P3 pentru a comanda cele 10 led-uri de pe bara de led-uri. Deci toți acești pini trebuie setați ca pini digitali de ieșire. Modul de ieșire va fi Push-Pull. Configurarea modului de ieșire al portului se face modificând valorile stocate în registrele P2MDOUT, respectiv P3MDOUT.
 - definiția registrului P2MDOUT se află în documentația microcontrolerului (fișierul C8051F04xRev1_4.pdf, pag. 218).
- configurează tensiunea de referință
 - referința de tensiune internă, la care este conectat convertorul analog-digital ADC2, este activată prin setarea bitilor BIAS (activează generatorul) și REFBE (activează bufferul referinței de tensiune) din registrul REF0CN.
 - definiția registrului REF0CN se află în documentația microcontrolerului (fișierul C8051F04xRev1_4.pdf, pag. 114).
- configurează convertorul analog-digital
 - pinul P1.0 este selectat ca intrare single-ended în multiplexorul analogic
 - convertorul se configurează pentru a porni conversia la setarea bitului AD2BUSY și pentru a avea castigul 1.

4 Desfasurarea lucrarii

1. Creați un nou proiect folosind fisierul Z:\Lab8051\Laborator2\lightIntensity.asm urmand pasii prezentati in Lucrarea de laborator nr. 1, la capitolul 2.2.
2. Executati aplicatia apasand butonul `Go` aflat in bara de butoane a IDE-ului si observati comportarea sistemului la schimbarea gradului de iluminare.
3. Opriti executia programului apasand butonul `Stop` si resetati sistemul apasand butonul `Reset`. Observati indicatorul albastru din stanga ferestrei de editare. De ce, in cazul unui reset, se reincepe executia de la acea linie?

Observatie: de aceasta data reset-ul a fost generat prin intermediul IDE-ului, insa exista mai multe surse de reset (au fost prezentate in Lucrarea de laborator nr. 1, capitolul 2.1.4.1.3.).

4. Incepeti executia programului pas cu pas, conform indicatiilor din capitolul 2.2.3.
5. Opriti executia pas cu pas inaintea instructiunilor de dezactivare a watchdog timer-ului. Vizualizati fereastra 8051 Contoller conform instructiunilor de la capitolul 2.2.3. Cautati in aceasta fereastra registrul de control al watchdog timer-ului (WDTCN). Consultati documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 171) pentru a stabili daca watchdog timer-ul este activ sau nu. Continuati rularea pas cu pas, executand si cele doua instructiuni ce au ca efect dezactivarea acestui timer. Ce s-a schimbat? Si-au indeplinit rolul aceste instructiuni?
6. Continuati rularea pas cu pas si vizualizati fereastra ADC2 conform instructiunilor de la capitolul 2.2.3. Observati valorile initiale ale registrelor AMX2CF, AMX2SL, ADC2CF, ADC2CN si valorile atribuite in procedura de initializare etichetata `initADC2` si confruntati-le cu definitiile acestor registre (fisierul C8051F04xRev1_4.pdf, pag. 95, pag. 97, pag. 98) si cu comentariile din codul sursa.
 - Care dintre registrele de mai sus determina castigul amplificatorului convertorului?
 - La ce valoare este setat acest castig?
 - Cum se preia tensiunea de la bornele celulei solare (simplu sau diferential)? Pe ce port (uri)?
 - Care dintre registrele de mai sus determina aceste setari?
7. Continuati rularea pas cu pas pana la intrarea in procedura `lightTest`. Aceasta procedura citeste efectiv valoarea tensiunii analogice de la intrarea ADC2, o converteste si proceseaza rezultatul (modificand valoarea acumulatorului). Rulati pas cu pas si observati cum setarea bitului AD2BUSY induce setarea bitului AD2INT. Daca intreruperile ar fi fost activate, setarea acestui bit ar fi generat o intrerupere de sfarsit de conversie. Modul cel mai simplu de a manipula convertorul analog-digital este de a astepta setarea bitului AD2INT (eveniment ce semnifica incheierea conversiei si stabilizarea rezultatului in registrul ADC2) urmand ca apoi sa se utilizeze valoarea convertita si stocata in ADC2. Instructiunea care asteapta setarea bitului AD2INT este `jnb AD2INT, $`.
8. Incercati optimizarea (din punct de vedere al timpului de executie) instructiunilor:

```
mov     B, #0x10;
div     AB
```
9. Continuati rularea pas cu pas pana ajungeti in subrutina `lightLedBar`. Acest bloc de program aprinde, pe rand, led-urile 0, 1, .. 10, in functie de valoarea stocata in acumulator. Vizualizati fereastra porturilor. Continuati rularea pas cu pas. Observati valorile din registrele P2 si P3 inaintea si dupa executia instructiunilor `setb`. Ce biti se modifica? De ce? Ce observati pe modulul de display?
10. Resetati microcontrolerul apasand butonul `Reset` din bara de butoane. Vizualizati fereastra stivei conform instructiunilor din capitolul 2.2.3. Ce valoare are indicatorul varfului stivei? Ce contine in acest moment stiva? Vizualizati si fereastra memoriei RAM (RAM Viewer) conform instructiunilor din capitolul 2.2.2. Faceti corelatia intre valoarea indicatorului de stiva si memoria RAM a controlerului.

Reincepeti rularea pas cu pas. Ce efect are instructiunea `ljmp main` asupra stivei? Explicati. Notati valoarea PC inaintea executiei instructiunii `call init`. Executati instructiunea. Ce se modifica in stiva? Dar in memoria RAM? Observati corelatia? Argumentati valorile SP, COUNT si STACK CONTENTS folosindu-va de explicatiile de la capitolul 2.2.3. De ce valoarea stocata in stiva este 0x0005? Cati bytes are instructiunea `call init` (Lucrarea de laborator nr. 1, capitolul 2.1.4.1.4.)? Care este adresa urmatoarei instructiuni tinand cont de valoarea PC-ului notata anterior? Ce conventie de stocare s-a folosit pentru acest numar de 16 biti (little endian/big endian)?

11. Continuati rularea pas cu pas pana la instructiunea `ret`. Ce se intampla cu stiva la executia ei? Unde s-a facut saltul (la ce adresa)?

12. Urmati pasii:

- modificati procedura `lightTest` astfel incat in interiorul ei sa apara o instructiune care insereaza un octet in stiva (`push`)
- salvati proiectul, compilati-l, asamblati-l si scrieti-l in memoria microcontrolerului conform indicatiilor din Lucrarea de laborator nr.1 (capitolul 2.2.)
- utilizati rularea “run to cursor” pentru a ajunge direct inaintea executiei instructiunii `call lightTest`. Observatie: programul a fost rulat pana in punctul respectiv; executia nu incepe efectiv din acel punct.
- notati valoarea PC-ului, calculati adresa instructiunii urmatoare si executati instructiunea. S-a modificat stiva in modul prevazut?
- executati instructiunea `push` nou inserata si observati noua valoare introdusa in stiva
- executati pas cu pas pana la instructiunea `ret`. Ce credeti ca se va intampla la executia ei? La ce adresa se va face saltul?
- executati instructiunea si observati rezultatul. Ce se intampla din acest moment cu programul?

Observati deci un comportament total inadecvat al executiei programului. Acest lucru se intampla daca stiva nu este manipulata corespunzator. Daca insa, inaintea intoarcerii din procedura (instructiunea `ret`), s-ar fi executat o instructiune `pop` rezultatul ar fi fost cel dorit. Nu ar fi aparut erori. Incercati sa introduceti si acesta instructiune.

O eroare si mai putin sesizabila poate aparea daca se modifica direct memoria RAM. Asa cum s-a vazut intre continutul stivei si memoria RAM exista o corespondenta definita prin valoarea SP. Daca memoria RAM este modificata in acele locuri comune cu stiva, implicit continutul stivei se modifica. Am vazut la un punct anterior ca si registrele de uz general sunt mapate tot in memoria RAM, deci prin simpla modificare a valorii unui registru se poate schimba continutul stivei. Incercati!

Pentru a se evita astfel de erori se obisnuieste ca la inceputul executiei programului sa se mute varful stivei (modificarea SP) undeva intr-o zona nefolosita a memoriei RAM. Astfel stiva nu va mai fi mapata incepand cu setul 1 de registre, ci altundeva in RAM, probabilitatea accesarii acelor adrese fiind mai mica.

13. Modificati proiectul astfel incat functionarea programului sa fie corecta.
14. Modificati programul astfel incat afisarea informatiei despre intensitatea luminoasa sa nu se mai faca pe bara de led-uri, ci cu ajutorul led-ului bicolor astfel:
 - led-ul sa ramana stins daca intensitatea luminoasa este foarte mica,
 - led-ul sa se aprinda verde daca intensitatea luminoasa este mica,
 - led-ul sa se aprinda rosu daca intensitatea luminoasa este mare,
 - led-ul sa se aprinda orange daca intensitatea luminoasa este foarte mare.

Indicatie

- led-ul bicolor este conectat la microcontroler conform ANEXEI 5.2.
15. Modificati programul astfel incat afisarea informatiei despre intensitatea luminoasa sa se faca si cu ajutorul afisajului cu 7 segmente astfel: daca pe bara de led-uri sunt aprinse X led-uri, atunci afisajul va arata cifra X. Indicatii:
 - afisajul cu 7 segmente este conectat la microcontroler conform ANEXEI 5.2.
 - fiecare din subrutinele prezentate in ANEXA 5.3 aprinde cate o cifra pe afisajul cu 7 segmente.

5 ANEXE

5.1 Codul aplicatiei lightIntensity

```

;-----
;
;
; FILE NAME      : lightIntensity.asm
; TARGET MCU     : C8051F040
; DESCRIPTION    : Light intensity detector.
;
; NOTES:
;
;-----

$include (c8051f040.inc)                ; Include register definition file.

;-----
; EQUATES
;-----

        LED1      equ      P3.7
        LED2      equ      P3.5
        LED3      equ      P3.3
        LED4      equ      P3.1
        LED5      equ      P3.0
        LED6      equ      P3.2
        LED7      equ      P3.4
        LED8      equ      P3.6
        LED9      equ      P2.7
        LED10     equ      P2.5

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg      AT 0x0000        ; Reset Vector
                ljmp      main            ; Locate a jump to the start of code
                                           ; at the reset vector.

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg      segment      CODE

                rseg      mainCodeSeg      ; Switch to this code segment.
                using     0                ; Specify register bank for the following
                                           ; program code.

main:            call      init            ; Initialization of all used SFR's
                mov       SFRPAGE, #ADC2_PAGE ; Use SFRs on the ADC2 Page

start:          ; Starting the algorithm

                call      lightTest        ; Tests the intensity of the light
                call      lightLedBar
                jmp       start            ; Start over again

;-----
; FUNCTION CODE
;-----

init:
initWatchDogTimer:  clr      EA                ; Disable global interrupts
                   mov       WDTCN, #0DEh      ; Disable Watch Dog Timer
                   mov       WDTCN, #0ADh
                   mov       SFRPAGE, #CONFIG_PAGE ; Use SFRs on the
                                           ; configuration Page

initCrossbar:      mov       XBR2, #0x40        ; Enable Crossbar

initIOPorts:       anl       P1MDIN, #0xFE      ; Configure P1.0 as analog input
                   orl       P2MDOUT, #0xA0      ; Set all the port pins connected to leds
                   orl       P3MDOUT, #0xFF      ; as output in push-pull mode
                   mov       P2, #0x00          ; Route P2 to ground
                   mov       P3, #0x00          ; Route P3 to ground
    
```

```
initVREF:      mov      SFRPAGE, #0x00      ; Use SFRs on the 0x00 Page
               mov      REF0CN, #0x03      ; ADC2 voltage reference from internal VREF
               ; Enable Bias Generator

initADC2:      mov      SFRPAGE, #ADC2_PAGE      ; Use SFRs on the ADC2 Page
               mov      AMX2CF, #0x00      ; Set AIN1.0 as single-ended input
               mov      AMX2SL, #0x00      ; Select AIN1.0 as input channel for
               ; the analog multiplexer
               mov      ADC2CF, #0xF9      ; Configure a *1 Gain
               mov      ADC2CN, #0x80      ; Enable ADC2; Continuous tracking;

               ret

lightTest:     clr      AD2INT
               setb     AD2BUSY      ; Force ADC2 to start a conversion
               jnb      AD2INT, $      ; Wait for ADC2 to finish the conversion

               mov      A, ADC2      ; Write the converted value in the accumulator
               mov      B, #0x18      ; Divides the value by 24.
               div      AB      ; [0x00, 0xFF] -> [0x00, 0x0A]
               ; the quotient is stored in A
               ; the remainder is stored in B

               ret

lightLedBar:   mov      P2, #0x00
               mov      P3, #0x00
               jz       endLight
               setb     LED1
               dec      A
               jz       endLight
               setb     LED2
               dec      A
               jz       endLight
               setb     LED3
               dec      A
               jz       endLight
               setb     LED4
               dec      A
               jz       endLight
               setb     LED5
               dec      A
               jz       endLight
               setb     LED6
               dec      A
               jz       endLight
               setb     LED7
               dec      A
               jz       endLight
               setb     LED8
               dec      A
               jz       endLight
               setb     LED9
               dec      A
               jz       endLight
               setb     LED10

endLight:      ret
```

```
;-----
; End of file.
```

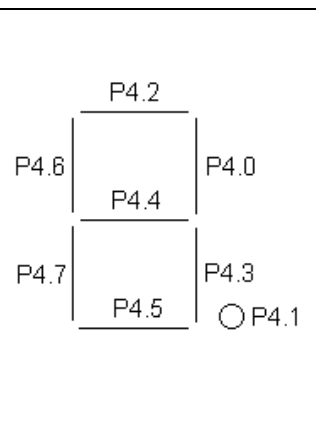
END

5.2 Conectarea modului de display la porturile microcontrolerului

Led-uri bicolore	
Led	Port
rosu #1	P2.2
verde #1	P2.0
rosu #2	P2.1
verde #2	P2.3

Bara de led-uri	
Led	Port
1	P3.7
2	P3.5
3	P3.3
4	P3.1
5	P3.0
6	P3.2
7	P3.4
8	P3.6
9	P2.7
10	P2.5

Afisaj cu 7 segmente	
Segment	Port
sus	P4.2
mijloc	P4.4
jos	P4.5
stanga-sus	P4.6
stanga-jos	P4.7
dreapta-sus	P4.0
dreapta-jos	P4.3
punct	P4.1
enable #1	P2.4
enable #2	P2.6



5.3 Codul subrutinelor pentru afisajul cu 7 segmente

```

lightZero:                mov             DIGIT, #0xED
                           ret

lightOne:                  mov             DIGIT, #0x09
                           ret

lightTwo:                  mov             DIGIT, #0xB5
                           ret

lightThree:                mov             DIGIT, #0x3D
                           ret

lightFour:                 mov             DIGIT, #0x59
                           ret

lightFive:                 mov             DIGIT, #0x7C
                           ret

lightSix:                  mov             DIGIT, #0xFC
                           ret

lightSeven:                mov             DIGIT, #0x0D
                           ret

lightEight:               mov             DIGIT, #0xFD
                           ret

lightNine:                mov             DIGIT, #0x7D
                           ret

lightError:               mov             DIGIT, #0xF4
                           ret
    
```

5.4 Solutie posibila #2

5.4.1 Descrierea modului hardware

Vom prezenta in continuare o implementare a unui sistem avand la baza placa de dezvoltare cu microcontrolerul C8051F040. Acesteia i se adauga doua module externe:

- modulul de intrare
 - contine o celula solara (dispozitivul de achizitie) ce se comporta ca o sursa de tensiune variabila (tensiunea oferita la iesirea celulei solare este direct proportionala cu intensitatea luminii incidente).
 - se conecteaza la microcontroler prin doua fire; unul este masa sistemului, iar celalalt este pentru tensiunea de iesire a celulei solare.
 - schema modului este prezentata in Fig. 5.4.1.a.

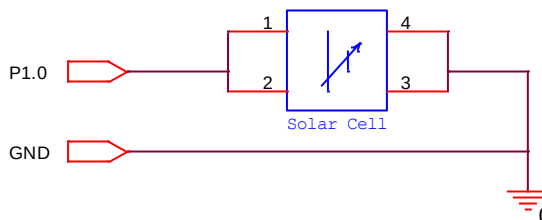


Fig. 5.4.1.a. Modulul de intrare

- modulul de iesire
 - contine, printre alte componente ce nu fac obiectul acestei lucrari, un dispozitiv de afisare (led bicolor rosu-verde).
 - se conecteaza la microcontroler printr-o magistrala cu 10 fire; dintre acestea sunt folosite doar trei: masa sistemului, anodul “rosu” al led-ului, anodul “verde” al ledului. Observatie: led-ul bicolor este un sistem ce incorporeaza in aceeasi capsula doua led-uri de culori diferite care impart acelasi catod.
 - Schema partii de modul ce priveste acest laborator este prezentata in figura Fig. 5.4.1.b.

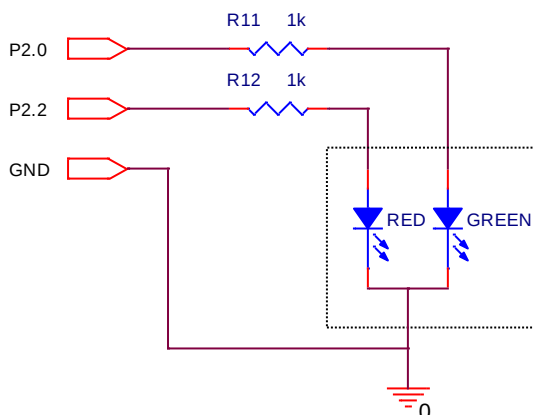


Fig. 5.4.1.b. Modulul de iesire

Dupa cum am spus celula solara se comporta ca o sursa de tensiune variabila cu valori intre 0 si 2.4 volti. Aceasta tensiune este introdusa in multiplexorul analogic prin pinul P1.0, modul de achizitie fiind simplu (nu diferential).

Valoarea digitala de la iesirea sistemului de conversie analog-digitala este data de formula:

$$\text{Cod} = V_i * (K/V_{\text{ref}}) * 256$$

In care:

- Cod este pe opt biti (intre 0 - 255 in zecimal)
- V_i este tensiunea de la intrarea multiplexorului analogic
- K este castigul amplificatorului programabil (0.5, 1, 2, 4)

- V_{ref} este tensiunea de referinta

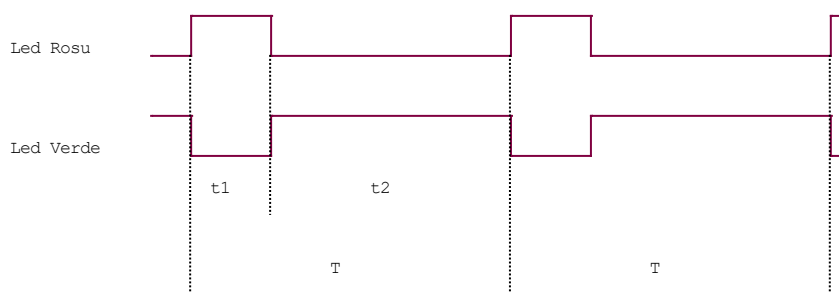
Observatie: codul convertit este un numar intre 0 si 255, deci $V_i * (K/V_{ref})$ trebuie sa aiba o valoare subunitara.

In aceste conditii se alege tensiunea de referinta V_{ref} egala cu 2.4V (se foloseste chiar referinta interna de tensiune a microcontrolerului), iar castigul amplificatorului programabil K egal cu 1. Pentru a obtine 2.4V la iesirea referintei interne de tensiune trebuie activat si amplificatorul de castig 2 (vezi capitolul 2.1.3.). Pentru a ruta referinta de tensiune pinii 5 si 6 din J22 trebuie conectati printr-un jumper.

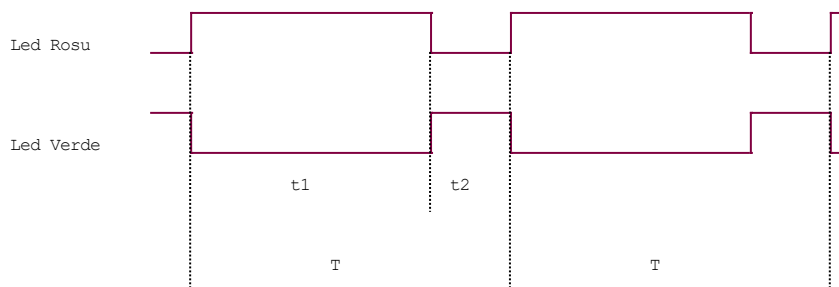
La iesirea sistemului se conecteaza pe pinii P2.0 si P2.2 un led bicolor. In mod normal un astfel de dispozitiv de afisare se poate afla in una din urmatoarele patru stari: stins complet, aprins rosu, aprins verde, aprins atat rosu cat si verde (obtinandu-se o culoare intermediara). Totusi, in aplicatia de fata s-a gasit o modalitate de afisare calitativa (cu mai multe stari) variind factorul de umplere al semnalelor aplicate celor doua led-uri.

Dupa cum ati vazut in lucrarea trecuta, sistemul vizual uman nu percepe palparea unui led cu o frecventa mare (mai mare decat frecventa critica). Acest fenomen se exploateaza si in aceasta lucrare, perioada semnalelor aplicate led-urilor fiind foarte scurta.

Intr-o astfel de perioada led-ul rosu se va aprinde un interval de timp t_1 , iar led-ul verde un interval de timp t_2 . Raportul acestor intervale dicteaza culoarea perceputa de sistemul vizual uman. Exemple:



Exemplantul 1



Exemplantul 2

In Exemplantul 1, culoarea perceputa de sistemul vizual uman este foarte aproape de verde, cu o nuanta rosiatica, in timp ce in Exemplantul 2, rosul este predominant.

Observatie: este necesar ca perioada T sa fie constanta.

Folosind aceasta metoda s-a ales sa se imparta perioada semnalului T in 16 cuante egale, led-ul bicolor putand sa se aprinda pe 16 nivele de culoare intre rosu si verde (verde pur va indica intuneric, iar rosu pur iluminare puternica).

5.4.2 Descrierea algoritmului

Observatie: programul respecta structura generala prezentata in Lucrarea nr.1, capitolul 3.2.2.

In solutia propusa in acest laborator programul va avea in mai multe rutine: `main`, `baseDelay`, `init`, `lightTest`, `redOn`, `redOff`, `greenOn`, `greenOff`.

Algoritmul consta intr-o bucla de durata constanta ce se autoapeleaza la infinit. Aceasta bucla are trei etape:

- se converteste tensiunea de la bornele celulei solare intr-o valoare digitala pe opt biti (intre 0 si 255).

- se imparte valoarea pe opt biti la 16 obtinandu-se un numar intre 0 si 15, reprezentand chiar nivelul de culoare
- se aprinde led-ul rosu un interval de timp (t_1) determinat de nivelul de culoare.
- se aprinde led-ul verde un interval de timp (t_2) determinat de nivelul de culoare.

Observatie: durata buclei este constanta, deci suma intervalelor de timp cat sunt aprinse ledurile ($T = t_1 + t_2$) este de asemenea constanta. Aceasta perioada este impartita in 16 cuante egale.

Subrutina `lightTest` face o conversie analog-digitala. Valoarea rezultata (intre 0 si 255) este impartita la 16, obtinandu-se un numar intre 0 si 15, reprezentand numarul de cuante din perioada semnalului cat va sta aprins led-ul rosu. Acest numar este stocat in registrul R6. Registrul R7 va stoca numarul de cuante cat va sta aprins led-ul verde ($15 - R6$).

Subrutina `baseDelay` este o rutina de intarziere care foloseste registrul R2 pe post de contor.

Rutina principala (`main`) apeleaza subrutina de initializare (`init`), apoi trece in zona de bucla infinita etichetata `start`. Aici se apeleaza rutina `lightTest` folosind apoi valorile stocate in R6 si R7. Codul etichetat `r` aprinde led-ul rosu daca este necesar (daca numarul de cuante cat trebuie sa stea aprins este nenul). Se executa apoi codul etichetat `redLoop` in care led-ul rosu este tinut aprins un interval $R6 * \text{baseDelay}$, dupa care este stins. In mod analog se parcurge codul etichetat `g`, apoi `greenLoop`, care va aprinde led-ul verde un interval $R7 * \text{baseDelay}$. In final, se executa un salt neconditionat la eticheta `start`.

Subrutina de initializare

- dezactiveaza intreruperile
 - pentru detalii, a se vedea explicatiile din Lucrarea nr. 1.
- dezactiveaza watchdog timer-ul
 - pentru detalii, a se vedea explicatiile din Lucrarea nr. 1.
- activeaza crossbar-ul
 - pentru detalii, a se vedea explicatiile din Lucrarea nr. 1.
- configureaza portul P1
 - aplicatia configureaza pinul 0 din portul P1 ca fiind pin de intrare analogica pentru convertorul analog-digital. Configurarea modului de intrare al portului se face modificand valoarea stocata in registrul P1MDIN.
 - definitia registrului P1MDIN se afla in documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 219).
- configureaza portul P2
 - aplicatia foloseste pinul 0 si pinul 2 din portul P2 pentru a comanda cele 2 led-uri (rosu si verde). Deci, pinul P2.0 si pinul P2.2 trebuie setati ca pini digitali de iesire. Modul de iesire va fi Push-Pull. Configurarea modului de iesire al portului se face modificand valoarea stocata in registrul P2MDOUT.
 - observatie: pentru a nu afecta celelalte circuite conectate cu panglica, toti pinii portului sunt rutati initial la masa.
 - definitia registrului P2MDOUT se afla in documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 218).
- configureaza tensiunea de referinta
 - referinta de tensiune interna, la care este conectat convertorul analog-digital ADC2, este activata prin setarea bitilor BIAS (activeaza generatorul) si REFBE (activeaza bufferul referintei de tensiune) din registrul REF0CN.
 - definitia registrului REF0CN se afla in documentatia microcontrolerului (fisierul C8051F04xRev1_4.pdf, pag. 114).
- configureaza convertorul analog-digital
 - pinul P1.0 este selectat ca intrare single-ended in multiplexorul analogic
 - convertorul se configureaza pentru a porni conversia la setarea bitului AD2BUSY si pentru a avea castigul 1.

Organigrama programului este prezentata in Fig. 5.4.2.

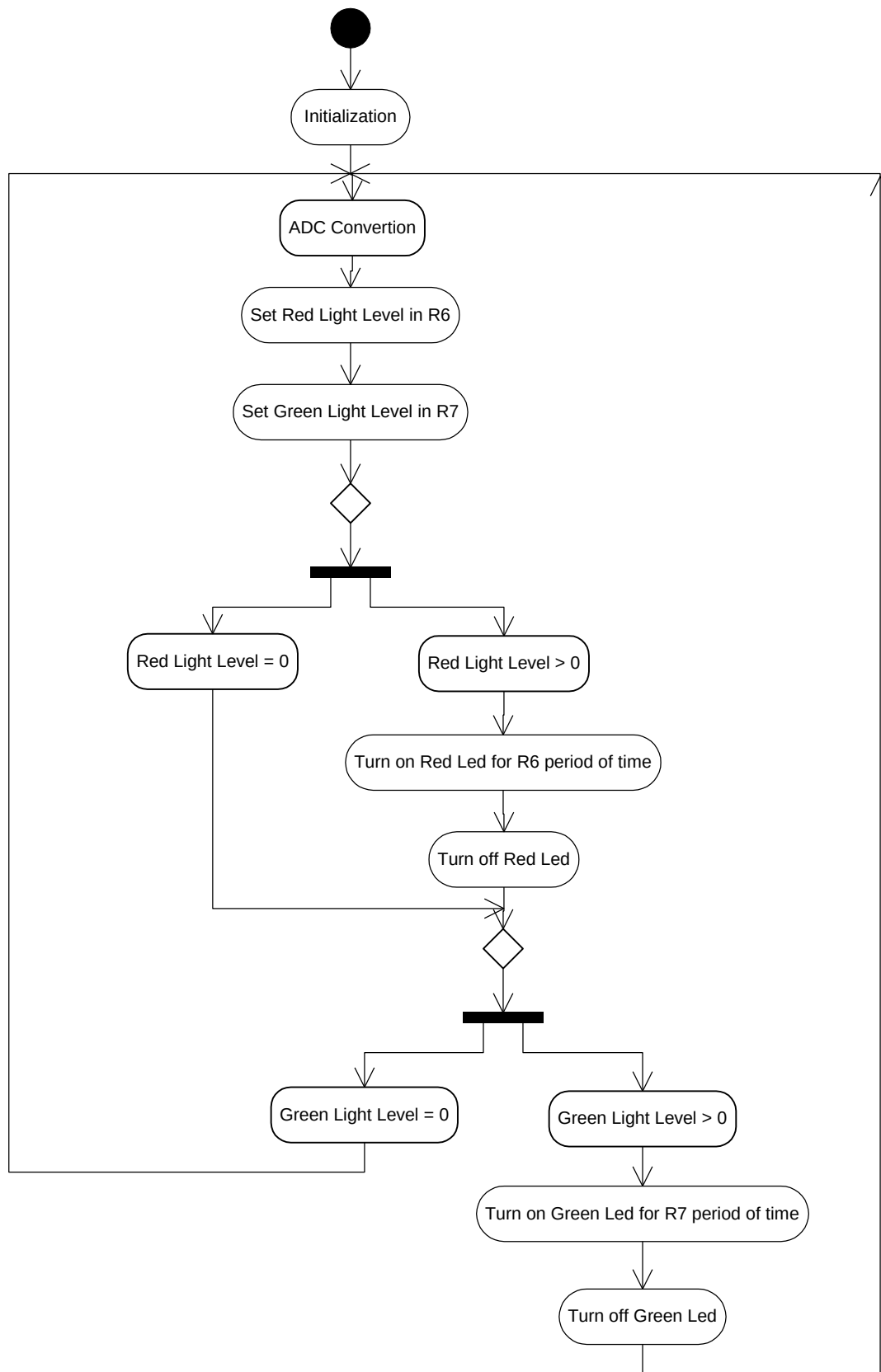


Fig. 5.4.2. Organigrama aplicatiei RedGreenLed

5.5 Codul aplicatiei RedGreenLed

```

#include (c8051f040.inc)                ; Include register definition file.

;-----
; EQUATES
;-----

        RED            equ        P2.2
        GREEN          equ        P2.0

;-----
; RESET and INTERRUPT VECTORS
;-----

                cseg        AT 0x0000        ; Reset Vector
                ljmp        main            ; Locate a jump to the start of code
                                           ; at the reset vector.

;-----
; MAIN PROGRAM CODE SEGMENT
;-----

mainCodeSeg    segment        CODE

                rseg        mainCodeSeg    ; Switch to this code segment.
                using      0              ; Specify register bank for the following
                                           ; program code.

main:
                acall      init           ; Initialization of all used SFR's
                mov        SFRPAGE, #ADC2_PAGE ; Use SFRs on the ADC2 Page

start:
                                           ; Starting the algorithm

                acall      lightTest      ; Tests the intensity of the light

        r:
                mov        A, R6
                jz         g
                acall      redOn
        redLoop:
                acall      baseDelay      ; Keeps the red LED lighted for
                dec        A              ; R6 base_delays
                jnz        redLoop
                acall      redOff         ; Red loop is over. Turn off the red LED

        g:
                mov        A, R7
                jz         start
                acall      greenOn
        greenLoop:
                acall      baseDelay      ; Keeps the green LED lit for
                dec        A              ; R7 base_delays
                jnz        greenLoop
                acall      greenOff       ; Green loop is over. Turn off the green LED

                sjmp       start         ; Start over again

;-----
; FUNCTION CODE
;-----

init:
initWatchDogTimer:    ...                ; Disable global interrupts
                                ...        ; Disable WatchDog Timer

initCrossbar:        ...                ; Enable Crossbar

initIOPorts:
                anl        P1MDIN, #0xFE    ; Configure P1.0 as analog input
                orl        P2MDOUT, #0x05    ; Set P2.0 and P2.2 (GREEN/RED) as digital
                                           ; output in push-pull mode.
                mov        P2, #0x00        ; Route P2 to ground

initVREF:
                mov        SFRPAGE, #0x00    ; Use SFRs on the 0x00 Page
                mov        REF0CN, #0x03    ; ADC2 voltage reference from internal VREF
                                           ; Enable Bias Generator

initADC2:
                mov        SFRPAGE, #ADC2_PAGE ; Use SFRs on the ADC2 Page
                mov        AMX2CF, #0x00    ; Set AIN1.0 as single-ended input
                mov        AMX2SL, #0x00    ; Select AIN1.0 as input channel for
                                           ; the analog multiplexer
    
```

```

                                mov      ADC2CF, #0xF9      ; Configure a*1 Gain
                                mov      ADC2CN, #0x80      ; Enable ADC2; Continuous tracking;
                                ret

baseDelay:                     mov      R2, #0x0F
                                djnz     R2, $
                                ret

redOn:                          ...                      ; Turn on the Red LED

redOff:                         ...                      ; Turn off the Red LED

greenOn:                        ...                      ; Turn on the Green LED

greenOff:                       ...                      ; Turn off the Green LED


lightTest:                     clr      AD2INT
                                setb     AD2BUSY            ; Force ADC2 to start a conversion
                                jnb      AD2INT, $          ; Wait for ADC2 to finish the conversion

                                mov      A, ADC2            ; Write the converted value in the accumulator
                                mov      B, #0x10           ; Divides the value by 16.
                                div      AB                ; [0x00, 0xFF] -> [0x00, 0x0F]
                                mov      R6, A              ; R6 stores the resulted value
                                mov      A, #0x0F
                                clr      C
                                subb     A, R6
                                mov      R7, A              ; R7 = 0x0F - R6
                                ret

;-----
; End of file.

END
```